PM Agent OS — Architecture

Oct 24, 2026 · @Dima

Design principles

A thin, always-responsive Chief of Staff (CoS) plans and delegates to pluggable modes; a few role-configured agents do the work through typed contracts; an independent quality loop reviews every output and turns failures into tested improvements. This evolves the earlier SERVICE.yaml / three-tier-skill framework from sequential agents into an async, hub-and-spoke system that anyone can extend by registering a mode.

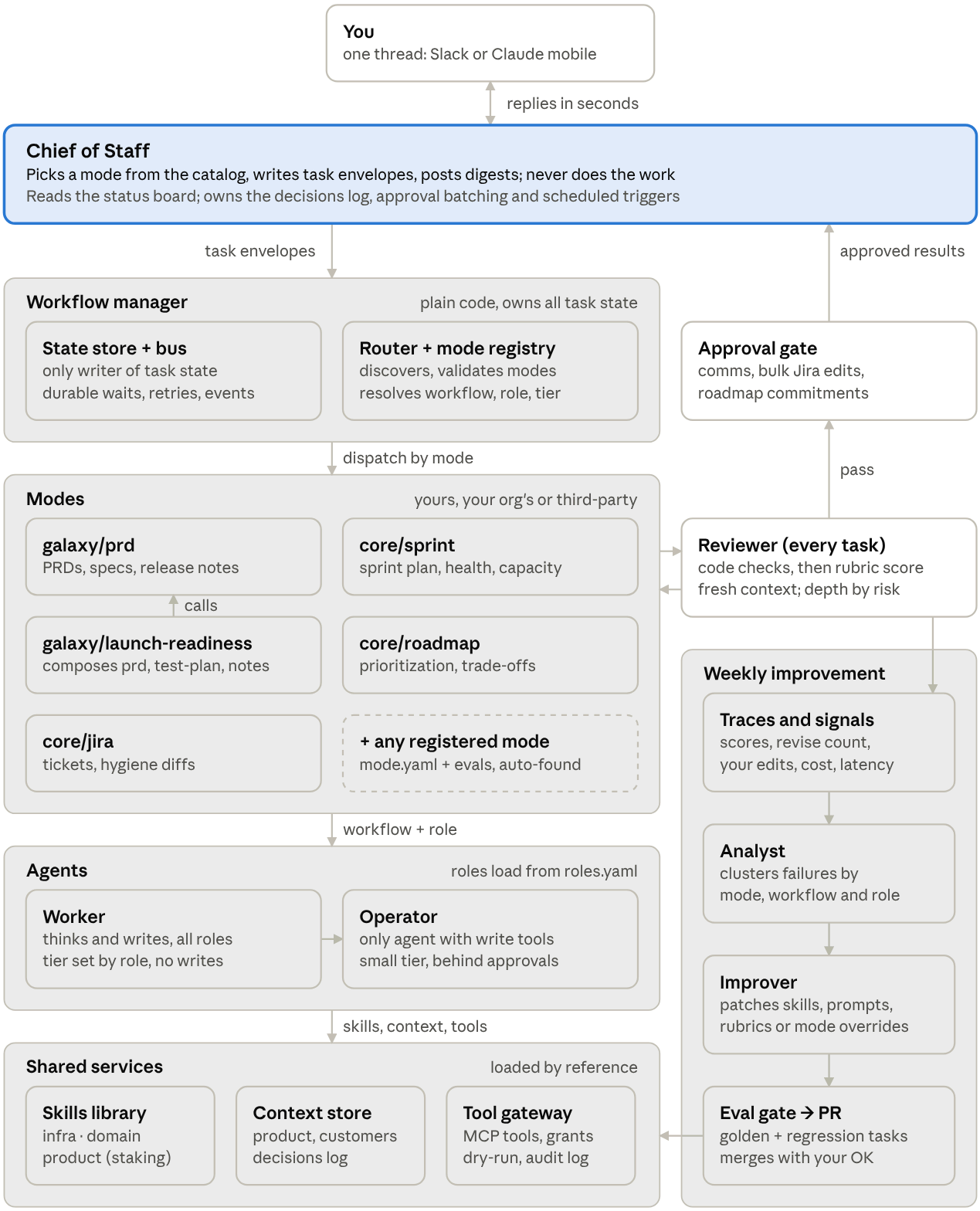
1. **The CoS thread stays free.** It acknowledges in seconds, dispatches work asynchronously, and only ever reads short result summaries — never full artifacts or tool output.
2. **Contracts over prompts.** Every hand-off is a versioned JSON envelope validated by code. Modes, roles and agents are swappable because the contract, not the prompt, is the interface.
3. **Code for deterministic work, models for judgment.** Routing, schema validation, Jira field mapping, budgets and retries are plain code. LLM calls are reserved for planning, writing and review.
4. **Right-size every call.** Each agent declares its model tier, tools and context budget in its manifest; escalation to a bigger model is an explicit, logged event.
5. **Systems of record are the truth.** Jira, the roadmap and the docs repo hold state; agents read them fresh instead of trusting memory or chat history.
6. **Nothing ships unreviewed; nothing irreversible ships unapproved.** A separate reviewer scores every output; external comms, bulk Jira edits and roadmap commitments wait for your yes.
7. **Every run is a training signal.** Traces, review scores and your edits feed an improvement loop that proposes changes and proves them on evals before they merge.

Behavior lives in modes and roles, not in agent count. Like pstack, the system grows by adding workflows and role config; only a handful of agents exist, split by permissions rather than by job.

Architecture

The CoS delegates to modes; agents do the work and every output is reviewed

Work flows down the left; reviewed results flow back up the right



PM agent OS · 4 layers, inline review, weekly improvement loop

Work flows down: your thread → CoS → the workflow manager → a registered mode, run by the Worker (thinking) or the Operator (writes), drawing on shared skills, context and tools. Modes can call other modes through the same bus. Every output goes right to the Reviewer (back to the specialist on a fail), up through the approval gate when needed, and reaches the CoS as a digest. Weekly, traces feed the improvement loop, whose eval-gated patches land in the shared skills and prompts.

Chief of Staff

The CoS is a conversation partner and dispatcher, not a worker: it turns what you say into work orders, hands them off, and comes back with digests. It runs on the mid tier (Sonnet-class) for fast replies and escalates to the top tier (Opus-class) only for multi-step planning it flags as ambiguous.

Its loop on every message

1. Classify intent: question, new work, status check, decision, or feedback.
2. Answer directly if it can from the status board and context index (no delegation for "what's blocking sprint 14?").
3. Otherwise pick a mode from the catalog and write one or more task envelopes, split into a dependency graph when the ask has parts, and post them to the task bus.
4. Reply immediately: what it dispatched, to whom, expected turnaround, anything it needs from you.
5. When results land, post a two-to-five line digest and surface only what needs a decision.

What it owns

- The status board: a read-only view of the workflow manager showing every open task, its state, owner and blocker.
- The decisions log: what you decided, when, and why — the shared memory every agent reads.
- Approval gates: it batches pending approvals into one message instead of interrupting you per item.
- Proactive cadence: scheduled triggers (Monday sprint health, Friday stakeholder update, daily Jira hygiene) that it dispatches without being asked.

What it never does

- Write a PRD, edit a ticket or call a work tool itself.
- Load full artifacts into its context; it reads the result summary and a link.
- Block on a worker; long work always runs async.

Context budget. System prompt, the compact mode catalog and the current status board are a stable, cached prefix. The rolling conversation is compacted into the decisions log so the thread can run for months without growing.

Workflow manager and state

A workflow manager — deterministic code — is the single writer of task state; agents report results and never set state, and the CoS only reads a view of it. It contains the state store, the task bus, the router and the mode registry.

State	Owner	Examples	Who reads it
Task and workflow state	Workflow manager	task states, dependencies, approval waits, child tasks of a composite mode	CoS status board, Reviewer, improvement loop
Domain state	Systems of record	Jira tickets, roadmap, published docs	agents, fresh at task start
Knowledge	Context store	decisions log, product and customer context, glossary	CoS, Worker roles

What it does

- Owns the state machine and applies every transition; a crashed or confused agent can't corrupt state.
- Runs composite modes as parent workflows that wait on their child tasks and merge the results.
- Holds approvals as durable wait states that survive restarts and prompt the CoS to remind you.
- Applies retries, timeouts and budget caps, with idempotency keys so a retry never double-creates tickets.
- Emits every transition as an event: the CoS digests and the improvement loop's traces both read this stream.

Build or adopt. Start with Postgres: a tasks table, a transitions table and a loop that advances runnable tasks — enough for single-step tasks. Once composite modes and multi-day approval waits arrive, adopt a durable workflow engine (Temporal, Restate, Inngest or DBOS) so waits, retries and resume-after-crash come built in, and each mode's workflow becomes a workflow definition. Either way the CoS, agents and modes see the same contract, so the swap doesn't touch them.

Task contract and dispatch

All work moves as a task envelope through the workflow manager. A deterministic router — code, no LLM — resolves the envelope's `mode` from the mode registry, then its `workflow` and `role` from that mode's manifest and the role sheet. Envelopes are generic verbs, so one shape serves every mode.

Task envelope (contracts/task.schema.json, v1)

```
{
  "id": "T-2026-1024-017",
  "verb": "draft",
  "mode": "galaxy/prd@^2",
  "workflow": "staking-prd",
  "role": "doc-writer",
  "inputs": {
    "goal": "PRD for multi-operator staking router, Solana first",
    "refs": ["jira:STK-412", "doc:staking-router-brief"]
  },
  "acceptance": ["covers custodian + bank personas", "has success metrics"],
  "risk": "medium",
  "approval": "none | before_publish | before_execute",
  "budget": { "max_usd": 1.50, "max_minutes": 20 },
  "depends_on": ["T-2026-1024-016"],
  "parent": "T-2026-1024-015",
  "reply_to": "cos"
}
```

`verb` is one of `plan`, `draft`, `analyze`, `execute`, `review`. `inputs` must validate against the mode's `accepts` schema; `workflow` and `role` are optional when the mode declares defaults.

Result envelope returns `status`, a ≤ 80-word `summary` for the CoS, `artifacts` as links, `self_check` against each acceptance item, `open_questions`, and a `trace_id` with tokens, cost and latency.

State machine: queued → running → in_review → (revise → running, at most 2 loops) → awaiting_approval → done. Any state can go to failed or blocked; blocked posts a question to the CoS.

Router responsibilities

- Resolve `mode` → pinned version → workflow → role → agent; an unknown or unregistered mode goes back to the CoS, never to a default.
- Validate `inputs` against the mode's `accepts` schema before anything runs.
- Pick model tier from the role, adjusted by `risk` (high risk can raise the tier, never lower it).
- Enforce budget and timeout; child envelopes from a composing mode draw down the parent's budget.
- Release a task only when its `depends_on` are done; run independent siblings in parallel.
- Route every finished task to review before it reaches the CoS.

Router and bus live inside the workflow manager. Jira or Linear is not the bus — they are systems of record the agents write to through the Operator.

Modes

A mode is the unit of extension: a packaged, versioned capability — yours, your org's or a third party's — that the registry discovers, validates and offers to the CoS, and that other modes can call.

What a mode contains — a folder or git repo with `mode.yaml`, its workflows, any roles it adds, rubrics and evals.

```
name: galaxy/launch-readiness
version: 1.2.0
contract: task@1
author: dima
summary: Readiness pack for a staking product launch
accepts: { product: string, launch_date: date }
returns: { pack: doc_ref, risks: list }
workflows: [readiness-pack]
default_role: doc-writer
calls: [galaxy/prd@^2, core/test-plan@^1, core/release-notes@^1]
grants: { jira: read, docs: write }
approval: before_publish
rubric: rubric.md
evals: evals/
```

Discovery and registration

1. The registry scans local `modes/` and any git sources you add, like a plugin marketplace.
2. It validates the manifest, contract version and the schemas of every mode in `calls`.
3. Admission runs the mode's evals in dry-run; no passing evals, no registration.
4. You approve the requested grants once, at install. A mode never executes side effects itself — writes go through the Operator and the approval gate.
5. The mode joins the CoS catalog as one line: name, summary, `accepts`.

How the CoS delegates — it picks from the catalog like pstack's router picks a workflow. When two modes match, precedence decides: your modes, then your org's, then third-party; still tied, the CoS asks you.

Composition

- A mode calls another by emitting a child envelope on the bus, never by direct call, so review, budget, tracing and approvals apply at every hop.
- Typed `returns` → `accepts` makes chaining safe; the registry checks compatibility at install and on every version bump.
- Guardrails: no cycles, a maximum call depth, and child budgets drawn from the parent's.

Third-party modes are namespaced (`author/mode`) and pinned by semver. Their outputs get the framework's baseline rubric plus their own. Improvements you make land in a local override layer, so upstream updates never erase them; you can send the patch upstream.

Agents and roles

Four agents do all the work, split by permissions; everything job-specific is a role in `roles.yaml`. New kinds of PM work arrive as roles or modes, never as new agents. Tiers: small = Haiku-class, mid = Sonnet-class, top = Opus-class.

Agent	Does	Tier	Tool grants
Chief of Staff	talks to you, picks modes, writes envelopes, posts digests	mid, top for hard planning	mode catalog, status board
Worker	plans, drafts and analyzes for every thinking role	set by the role	read-only tools, draft documents
Operator	executes side effects: Jira writes, publishing, sending comms	small	the only write grants, behind the approval gate
Reviewer	scores every output against baseline + mode + role rubrics	scaled by risk	read-only

Role sheet — a role is config the Worker loads: persona, skills, context files, rubric and tier. Modes name a default role and may ship extra roles of their own.

Role	Default tier	Key skills	Used by modes like
roadmap-strategist	top	prioritization, trade-off memos	`core/roadmap`
sprint-planner	mid	capacity math, sprint health	`core/sprint`
jira-curator	small	Jira query patterns, hygiene rules	`core/jira`
doc-writer	mid	PRD structure, staking PRD, release notes	`galaxy/prd`, `galaxy/launch-readiness`
tech-architect	top	API design, staking router context	`core/tech-spec`

Role	Default tier	Key skills	Used by modes like
qa-planner	mid	test plans, acceptance criteria	`core/test-plan`
research-analyst	mid	market scans, feedback synthesis	`core/research`
communicator	mid	status updates, stakeholder notes	`core/comms`

The permission line. Roles may share the Worker only because the Worker holds no write or send rights. Any role's output that must change a system — new tickets, a sent update — becomes an `execute` envelope for the Operator.

Keep quality visible per role. Rubrics and evals stay keyed by mode, workflow and role, and every trace logs all three plus the agent, so the improvement loop can tell that the PRD workflow is weak, not just that the Worker is.

Shared services

Three services sit under every agent, so roles and modes stay small and knowledge is written once.

Skills library (three tiers, unchanged from the current framework)

- *Infrastructure:* envelope I/O, Jira query patterns, doc formatting, citation rules, self-check.
- *Domain:* PRD structure, prioritization methods (RICE, cost of delay), sprint capacity math, release-note style.
- *Product:* staking-specific — the institutional staking PRD skill, custodian/bank/FI personas, Ethereum and Solana staking mechanics, operator glossary.

Roles and modes load skills by reference from their manifests, and only the sections a task type needs, which keeps per-call context small.

Context store — curated, versioned files rather than a vector dump: `product.md`, `customers.md`, `glossary.md`, `roadmap.yaml`, `decisions.log`, plus a generated one-line-per-file index. Agents read the index, then pull only the files they need. The Analyst refreshes summaries nightly.

Tool gateway — all external systems via MCP servers (Jira, Confluence or Notion, Google Drive, Slack, GitHub, calendar), wrapped by one gateway that enforces:

- per-agent grants from the manifest (read, write, draft-only);
- dry-run mode that returns a diff instead of executing;
- rate limits and idempotency keys so retries never double-create tickets;
- a full audit log tied to `trace_id`.

Review and continuous improvement

Quality runs on two clocks: an inline Reviewer gates every output in minutes, and a weekly improvement loop turns patterns of failure into tested changes.

Inline review (every task)

1. Code checks first, free and instant: schema valid, links resolve, required sections present, Jira fields legal.
2. Reviewer agent scores the output against the framework's baseline rubric plus the mode's and role's `rubric.md` and the envelope's acceptance list. It runs in a fresh context and sees only the brief and the artifact, never the producer's reasoning.
3. Pass → awaiting_approval or done. Fail → one revise loop with the reviewer's notes; a second failure escalates to the CoS with both versions.
4. Review depth scales with risk: high-risk and external-facing work gets a top-tier review; low-risk hygiene work gets a small-tier spot check on a sample.

Signals captured per task — rubric score, revise count, your edits to the artifact after delivery (diff size), approval rejections, budget or timeout breaches, latency, cost. Your edits are the strongest signal: they show exactly where the agent missed.

Weekly improvement loop (batch, off-peak)

1. *Analyst* clusters failing traces by mode, workflow, role and failure type.
2. *Improver* (the existing designer-agent pattern) proposes a concrete patch: a skill edit, rubric line, prompt change, routing rule, tier change, or a local override on a third-party mode.
3. Patch runs against that mode's and role's eval sets — golden tasks plus recent real failures added as regression cases.
4. Only patches that improve the target metric without regressing others open a PR; you approve, the manifest version bumps, the change log records why.

5. Cost and latency trends are reviewed in the same pass: a role consistently scoring well becomes a candidate for a smaller tier.

Cost, speed and accuracy levers

Most savings come from not calling a model at all and from small, cached contexts; most accuracy comes from contracts, fresh-context review and your edit signal.

Lever	How it works	Mainly improves
Model tiering	Small tier for Jira ops, classification and formatting; mid for writing and planning; top only for strategy, architecture and high-risk review	cost, speed
Prompt caching	Stable prefix (system prompt, skills, product context) first, variable task last; caches stay warm across a burst of tasks	cost, speed
Brief, don't forward	CoS sends a short brief plus refs; agents fetch what they need instead of receiving the chat history	cost, accuracy
Code before models	Routing, validation, Jira field mapping and dependency checks in code	cost, speed, accuracy
Parallel fan-out	Independent envelopes in a task graph run at the same time	speed
Async CoS	Ack in seconds, results stream back as digests	speed (perceived)
Batch API	Nightly hygiene sweeps, weekly analysis and eval runs at the discounted batch rate	cost
Risk-scaled review	Full top-tier review for external or irreversible work; sampled small-tier checks for routine work	cost, accuracy
Structured outputs	JSON schemas for every result; malformed output is retried by code, not by you	accuracy
Fresh-context reviewer	Reviewer never sees the producer's reasoning, so it can't inherit its mistakes	accuracy

Lever	How it works	Mainly improves
Systems of record	Agents re-read Jira and the roadmap at task start instead of relying on stale context	accuracy
Budget guards	Per-task cost and time caps in the envelope; breaches stop the task and report	cost

Measure from day one: cost per task type, p50/p90 latency, first-pass review rate, and your edit rate. These four numbers drive every tier and routing change in the improvement loop.

Extending and maintaining

Extending is registering: drop a mode folder or add a git source and the registry picks it up; add a role entry for a new kind of work. No orchestrator code or CoS prompt changes.

Repo layout

```
pm-os/
  cos/            prompt.md, precedence.yaml, digest_templates/
  agents/         worker/, operator/, reviewer/   (stable, rarely touched)
  roles.yaml      role sheet: persona, skills, context, rubric, tier
  modes/
    core/prd/     mode.yaml, workflows/, rubric.md, evals/
    galaxy/launch-readiness/ ...
  sources.yaml    git sources for org and third-party modes
  overrides/      local patches on third-party modes
  skills/
    infra/  domain/  product/
  context/        product.md, customers.md, glossary.md, roadmap.yaml,
decisions.log, INDEX.md
  contracts/      task.schema.json, result.schema.json, mode.schema.json
(semver)
  runtime/        workflow/ (state store, bus, workflows), router/,
registry/, gateway/
  quality/        analyst/, improver/, evals/shared/, baseline_rubric.md
  traces/         one JSONL line per task
```

Role entry (roles.yaml)

```
doc-writer:
  version: 1.4.0
  agent: worker
  tier: { default: mid, escalate_to: top, escalate_when: risk == high }
  context_budget_tokens: 40000
  skills: [infra/envelope-io, infra/citations, domain/prd-structure,
product/staking-prd]
  context: [product.md, customers.md, glossary.md]
  rubric: rubrics/doc-writer.md
  evals: evals/doc-writer/
```

Rules that keep it maintainable

- New work → a new mode or role; a new agent only when it needs different permissions or isolation.
- One mode = one capability with typed inputs and outputs; bigger capabilities compose smaller modes.
- Contracts are versioned; modes declare the contract version they accept, so a schema change never breaks a registered mode.
- No evals, no registration — for your modes and third-party ones alike.
- Skills are shared, never copied into prompts; a fix in one skill improves every role and mode that uses it.
- Model names live only in a tier map (`small/mid/top → model id`), so a model upgrade is one line plus an eval run.

Running it on what you already use: the CoS as a Claude Code or Agent SDK session (reachable from Slack for the mobile thread), the four agents as subagents, modes packaged like plugins with skill folders for their workflows, MCP servers for the tools, and hooks on task and subagent completion to write traces.

Build plan

Ship the spine first with two agents, prove the loop end to end, then add specialists one at a time behind evals.

1. **Spine.** Contracts (task, result, mode), workflow manager on Postgres (state store, bus, router, mode registry), tool gateway in dry-run, trace logging, CoS with status board and decisions log.

    - Exit: a request from your phone becomes a tracked task and a digest comes back without the CoS doing the work.
2. **Worker, Operator, Reviewer and two modes.** core/jira and galaxy/prd with their roles (reusing the staking PRD skill), inline review with rubrics, approval gates.
    - Exit: PRDs and ticket batches go from ask to approved without manual rework most of the time; edit rate is being logged.
3. **Planning layer.** core/sprint and core/roadmap modes, plus scheduled triggers for sprint health and weekly stakeholder updates via core/comms.
    - Exit: Monday sprint health arrives unprompted and matches Jira.
4. **Improvement loop.** Analyst and Improver on batch, eval sets per agent, PR-based patches.
    - Exit: at least one merged patch shows a measured gain on its eval set.
5. **Breadth.** Composite modes such as galaxy/launch-readiness, tech-spec, test-plan and research modes, then git sources for org and third-party modes; tier tuning from real cost and quality data.

Open questions

- [ ] Which tracker and docs tool are the systems of record (Jira + Confluence, or Notion/Google Docs)?
- [ ] Where should the CoS thread live day to day — Slack, Claude mobile, or both?
- [ ] Which actions can run under standing auto-approval from the start?
- [ ] Who may publish modes to the org source, and who approves their grants?